

Video Intro POO

Vocabulaire : Classe, objet comme instance de classe,

Attribut : composante des objets de la classe qui définissent l'état des objets

Propriété en Python : droits de manipulation des attributs (lecture seule, accès non autorisé en dehors de la classe)

Méthode : fonctions permettant de lire ou modifier les objets de la classe

Intérêt de l'objet : structuration du code : découplage des modules (rendus autonomes), réutilisabilité, maintenabilité

26 min < : exemple de la classe str avec le formatage

Video Classes et Attributs

Définition d'une classe, l'instanciation, le constructeur avec passage de paramètres, les variables de classes (compteur d'objets existants)

Video Méthodes

Méthode d'objet (fonction agissant sur une instance de la classe) → 8min42,

méthode de classe (fonction agissant sur la classe), méthode statique (indépendante de la classe mais liée à une classe dans sa définition)



La programmation orientée objet (POO) est un concept de programmation puissant qui permet de structurer ses programmes d'une manière nouvelle.

Une **classe** définit et nomme une structure de données (données que l'on nomme « attributs ») qui vient s'ajouter aux structures de base du langage (les types natifs du langage comme les entiers, les listes, ...)

L'état des objets issus de cette classe (on dit que ces objets sont des **instances** de cette classe) est défini par la valeur de ses « **attributs** » et le comportement de ces objets est défini par des « **méthodes** » qui permettent *d'animer* cet objet c'est-à-dire de faire évoluer son état en modifiant ses attributs.

Rq 1. Les attributs s'apparentent donc à des variables, composantes de l'objet, et les méthodes à des fonctions attachées à l'objet, qui agissent sur ces variables ou qui les utilisent en lecture.

Rq 2. Tous les objets instanciés à partir d'une même classe sont donc comme construits dans le « moule » de la classe, qui en fixe les attributs communs et les méthodes communes.



Dans ce chapitre on utilisera les mots objet ou instance pour désigner la même chose.

Rq 3. en Python, tout est objet. Une variable de type **int** est en fait un objet de type **int**, donc construit à partir de la classe **int**. Pareil pour les **float** et **string**. Mais également pour les **list**, **tuple**, **dict**, etc. Voilà pourquoi nous avons déjà rencontré de nombreuses notations et mots de vocabulaire associés à la POO.

Ex 1 : De classes déjà rencontrées :

```
>>> L1=[3,2,4]
>>> L2=[30,20,40,50]
>>> type(L1)
<class 'list'>
>>> type(L2)
<class 'list'>
L1.sort()
>>> L1
[2, 3, 4]
>>> L2.append(25)
>>> L2
[30, 20, 40, 15, 25]
```

Dans la séquence ci-dessous, repérer les classes, les objets et les méthodes

Rq 4. De même, les widgets de TKInter sont des objets. (cf la classe des **button**)

Rq 5. L'utilisation de classes évite l'utilisation de variables globales en créant ce qu'on appelle un espace de noms, propre à chaque objet, permettant d'y encapsuler des attributs et des méthodes. La POO permet donc de rédiger non seulement du code plus compact, mais aussi plus **réutilisable**.
On peut réaliser des programmes extrêmement complexes uniquement en utilisant des classes préexistantes.



Les idées sous-tendant le **paradigme** objet datent des années 60. Mais il faudra attendre le début des années 70 et la mise au point du langage Smalltalk pour que le paradigme objet gagne en popularité chez les informaticiens. Aujourd'hui de nombreux langages permettent d'utiliser le paradigme objet : C++, Java,...

Rq 6. De plus, la POO amène de nouveaux concepts (hors programme de terminale) tels que le *polymorphisme* (capacité à redéfinir le comportement des opérateurs), ou bien encore l'héritage (capacité à définir une classe à partir d'une classe préexistante et d'y ajouter de nouvelles fonctionnalités).



Malgré tous ces avantages, la POO peut paraître difficile à aborder pour le débutant pressé d'en découdre avec le codage : elle exige une réflexion préalable qui participe de la conception d'un logiciel. Elle nécessite beaucoup de pratique. Bien structurer ses programmes en POO est un travail d'artiste (comme peut l'être aussi la rédaction d'une preuve de maths). Il existe des langages qui formalisent la construction de programmes orientés objets, par exemple le langage UML (on parle là d'outils de génie logiciel)



Pour un exemple à base de géométrie et la classe des points : <https://courspython.com/classes-et-objets.html>

I. LA CLASSE ET LES INSTANCES

La création d'une classe en python commence toujours par le mot **class**. Toutes les instructions relevant de la classe seront repérées par l'indentation.

```
class Le_Nom_De_Ma_Classe:
    #instructions de la classe
    #La définition de la classe est terminée.
```

Ex 2 : Création d'une classe Personnage (qui sera dans un premier temps une coquille vide) et, à partir de cette classe, création de deux instances : bilbo et gollum.

Exo 1. Editer, analyser et tester ce code

```
class Personnage:
    pass

gollum = personnage()
bilbo = personnage()
```

Rq 7. Il n'est pas possible de créer une classe totalement vide (pas d'attributs, pas de méthodes), on utilise donc ici l'instruction **pass** qui ne fait rien.
On crée ensuite deux instances de la classe **Personnage** : gollum et bilbo.
A ce stade, cette classe ne sert à rien et les deux objets créés ne peuvent rien faire.

II. LES ATTRIBUTS ET LE CONSTRUCTEUR

Exo 2. Création d'un attribut via :

1. On utilise



le mot réservé du langage **__init__** qui désigne une méthode particulière : le constructeur d'un objet qui permet de construire et initialiser une instance de la classe ,

- et le mot **self** qui désigne l'instance elle-même, toujours premier paramètre de chaque méthode.
- Ici, un deuxième paramètre fournissant la valeur du nombre de vies du personnage à sa création.

```
class Personnage:
    def __init__(self, nbre_vies):
        self.vie=nbre_vies
```

nom de la classe
attributs
méthodes


```
bilbo=Personnage(15)
```

nom de l'objet
attributs
méthodes


```
gollum=Personnage(20)
```

nom de l'objet
attributs
méthodes

Le constructeur **__init__()** est une méthode automatiquement exécutée au moment de la création d'une instance.



Une méthode, comme une fonction, peut prendre des paramètres. Le passage de paramètres de la méthode **__init__()** se fait au moment de la création de l'instance.

```
gollum=Personnage(20)
```



Ici le paramètre formel **self** est associé au paramètre effectif **gollum** et le paramètre **nbre_vies** est associé au paramètre effectif 20.

On accède aux attributs de l'objet **bilbo** avec la notation **bilbo.a** où **a** est le nom de l'attribut visé.

- Utiliser la console Python pour vérifier que **gollum.vie** est égal à 20 et **bilbo.vie** est égal à 15

Rq 8. comme pour n'importe quelle fonction, il est possible de passer plusieurs arguments à la méthode **__init__()**.

Rq 9. Dans la terminologie Python, les attributs sont aussi appelés *variables d'instance*.

Même si *Python* permet la création dynamique de variables d'instances (tout comme il permet la création et le typage dynamique de variables au contraire d'autres langages plus contraignants), on s'imposera la règle suivante : chaque objet au moment de sa création se voit doté des attributs prévus pour sa classe et aucun autre ne lui sera ajouté par la suite. C'est le point de vue du paradigme objet classique.

III. LES METHODES (D'INSTANCE):

Exo 3. Saisir, analyser et tester ce code

```

class Personnage:
    def __init__(self, nbre_vies):
        self.vie= nbre_vies

    def get_etat (self):
        return self.vie

    def blesser (self):
        self.vie=self.vie-1

gollum = Personnage(20)
bilbo = Personnage(15)

```

1. Pour tester ce programme, dans la console, saisir successivement les instructions suivantes :

```

>>>gollum.get_etat()
>>>bilbo.get_etat()
>>>gollum.blesser()
>>>gollum.get_etat()
>>>bilbo.blesser()
>>>bilbo.get_etat()

```

On appelle la méthode **m(self, param1, param2,...)** de l'objet **bilbo** (instance de la classe **Personnage**) avec des valeurs de paramètres **p1,p2,...** en utilisant la notation **bilbo.m(p1,p2,...)** où **p1,p2,...** sont les valeurs effectives données aux paramètres formels **param1, param2,...**.

Rq 10. Pour faire le lien avec l'appel classique de fonction, on peut évoquer l'appel possible équivalent de cette méthode par la notation : **Personnage.m(bilbo, p1, p2,...)**

2. ou bien directement dans le programme, en accédant explicitement à l'attribut :


```

bilbo = Personnage(15)
print("bilbo a {} points de vie".format(bilbo.vie))

```
3. ou bien directement dans le programme, en utilisant la méthode **get_etat()** (dite *accesseur* ou *getter* (en anglais)) :


```

gollum = Personnage(20)
print("gollum a {} points de vie".format(gollum.get_etat()))

```
4. Les personnages peuvent boire une potion qui leur ajoute un point de vie.
Ajouter une méthode **boire_potion()** qui ajoute un point de vie au personnage instancié
Tester.
5. Selon le type d'attaque subie, le personnage peut perdre plus ou moins de points de vie avec un certain indéterminisme (aspect aléatoire). Pour tenir compte de cet élément, il est possible d'ajouter un paramètre à la méthode **blesser**, **max_blessure**. La blessure subie est alors une perte de points de vie aléatoire entre 1 et **max_blessure**
Coder et Tester.

6. Editer et analyser le code suivant (noter en particulier les nouveaux attributs et les nouvelles méthodes, et décrire le comportement de la fonction **game()**):

```
import random

class Personnage:
    def __init__(self,nom,nbre_vies):
        self.vie=nbre_vies
        self.nom=nom
    def __str__(self):
        return("{} a {} points de vie".format(self.nom,self.vie))

    def get_etat (self):
        return self.vie

    def blesser (self, max_blessures):
        nbPoint = random.randint(1,max_blessures)
        self.vie=self.vie-nbPoint

def game():
    bilbo=Personnage("Bilbo",15)
    gollum=Personnage("Gollum",20)
    print(str(bilbo))
    print(str(gollum))
    while bilbo.get_etat(>0 and gollum.get_etat(>0 :
        bilbo.blesser(5)
        gollum.blesser(5)
        print("combat : "+ str(bilbo)+str(gollum))
        if bilbo.get_etat(<=0 and gollum.get_etat(>0:
            msg = "Gollum est vainqueur, il lui reste encore {} points alors que Bilbo est mort".format(gollum.get_etat())
        elif gollum.get_etat(<=0 and bilbo.get_etat(>0:
            msg = "Bilbo est vainqueur, il lui reste encore {} points alors que Gollum est mort".format(bilbo.get_etat())
        else :
            msg = "Les deux combattants sont morts en même temps"
    return msg
```

classe
attributs

méthodes

objets
attributs

méthodes

Rq 11. On voit ici qu'il est possible d'utiliser le paradigme objet et le paradigme impératif dans un même programme.

Rq 12. La méthode `__str__()` (mot réservé *Python*) bénéficie d'un mode d'appel allégé : `str(bilbo)` au lieu de `bilbo.__str__()`

7. Améliorer le programme en modifiant des méthodes ou en implémentant vos propres méthodes.

Rq 13. Sur l'encapsulation des objets :

Dans la philosophie objet, l'accès aux objets d'une classe se fait essentiellement via les méthodes, les attributs étant considérés comme relevant du détail d'implémentation. Les méthodes constituent donc l'interface de la classe permettant l'accès aux instances à tout développeur utilisateur.

IV. ATTRIBUTS ET METHODES DE CLASSES

1. ATTRIBUTS DE CLASSES

Une classe permet la définition d'attributs dits de classe (et non attributs d'instances) dont la valeur est attachée à la classe elle-même et **partagée** par toutes les instances de la classe

Ex 3 :

```
class Personnage:
    #attributs de classe
    max_vies=30
    nb_personnages=0
    def __init__(self,nom,nbre_vies):
        self.vie=nbre_vies
        self.nom=nom
        Personnage.nb_personnages+=1 #accès à un attribut de classe

    def __str__(self):
        return("{} a {} points de vie".format(self.nom,self.vie))

bob=Personnage("Bob",15)
print(str(bob))
print("attribut de classe max_vies = {},vu de Bob : {}".format(
    Personnage.max_vies, bob.max_vies))
print("attribut de classe nb_personnages = {},vu de Bob : {}".format(
    Personnage.nb_personnages, bob.nb_personnages))
gandalf=Personnage("Gandalf",30)
print(str(gandalf))
print("attribut de classe max_vies = {},vu de Gandalf : {}".format(
    Personnage.max_vies, gandalf.max_vies))
print("attribut de classe nb_personnages = {},vu de Gandalf : {}".format(
    Personnage.nb_personnages, gandalf.nb_personnages))
```



[personnage_décompté.py](#)

2. METHODES DE CLASSE (Non essentiel...)



Non essentiel... voir [Video Méthodes](#)

Une **méthode de classe** (ou méthode **statique**) ne s'applique pas à un objet en particulier. Elles sont parfois pertinentes pour réaliser des fonctions auxiliaires ne travaillant pas directement sur les objets de la classe ou qui ne les modifient pas.

Ex 4 : Une méthode qui compare les formes respectives de deux personnages

```
def le_plus_en_forme(p1,p2):
    if p1.nbre_vies>p2.nbre_vies : return p1
    return p2
```



Ces méthodes de classe sont définies dans le corps de la classe **Personnage** sans utiliser le paramètre **self**



Pour appeler des méthodes de classe, on peut utiliser la méthode d'accès direct en préfixant par le nom de la classe :

```
>>> le_plus_fort = Personnage.le_plus_en_forme(bilbo,gandalf)
```

Rq 14. Les méthodes de classe rendent les mêmes services que des fonctions qui seraient définies à l'extérieur de la classe. C'est donc une notion non essentielle ici.

V. Exos

Exo 4. Définir une classe Chrono pour représenter le temps. Cette classe possède trois attributs, heures, minutes et secondes, entiers positifs ou nuls, minutes $\in [0,59]$ et secondes $\in [0,59]$

a. Ecrire le constructeur de cette classe. (les paramètres du constructeur doivent être des entiers positifs ou nuls sans autres contraintes.)

b. Ajouter une méthode `__str__()` qui renvoie une chaîne de caractères de la forme « 12h 34m 55s ».

```
>>> t=Chrono(21,34,55)
>>> str(t)
'21h 34m 55s'
```

c. Ajouter la méthode `clone()` qui renvoie un nouveau chrono égal à l'ancien.

```
>>> u=t.clone()
>>> str(u)
'21h 34m 55s'
```

d. Ajouter la méthode `avance()` qui prend un temps t exprimé en secondes et avance le chrono instancié de t .

e. Ajouter les méthodes `egale()` qui prend un deuxième chrono en argument et renvoie un booléen indiquant si les deux chronos sont égaux.

```
>>> t.egale(u)
True
>>> t.avance(5)
>>> str(t)
'21h 35m 0s'
>>> t.egale(u)
False
```

f. Reprendre les questions précédentes en modifiant l'implémentation : la classe Chrono est maintenant constituée d'un seul attribut exprimé en secondes.

Exo 5. Définir une classe Point pour représenter un point dans le plan. Cette classe possède trois attributs : nom (qui est une chaîne désignant le nom du point), x et y (qui désignent l'abscisse et l'ordonnée dans un repère).

- a. Ecrire le constructeur de cette classe.
- b. Ajouter une méthode `__str__()` qui renvoie une chaîne de caractères de la forme « A(1.3 ;3.5) »



La consigne ne fait pas apparaître le **self** dans l'interface (elle est implicite dans la consigne, elle est en revanche explicite dans l'implémentation *Python*)

- c. Ajouter une méthode `distance_origine()` qui renvoie la distance à l'origine du point.
- d. Ajouter une méthode `deplace(dx,dy)` qui translate le point instancié d'un vecteur $\vec{u} \begin{pmatrix} dx \\ dy \end{pmatrix}$. Cette méthode **renvoie le point image** (une instance de la classe Point) du point instancié.
- e. Ajouter une méthode `__eq__(p)` qui renvoie un booléen précisant si le point instancié et un autre point p passé en paramètre sont confondus et une méthode `distance(p)` qui renvoie la distance du point instancié à un autre point p passé en paramètre.



On peut alors tester si deux points A et B sont confondus en utilisant « **A==B** », et *Python* fera alors appel à **A.__eq__(B)**

- f. Ajouter une méthode `symetrique_O()` qui renvoie l'image du point instancié par symétrie par rapport à l'origine O – cette méthode ne déplace pas le point passé en paramètre -, une méthode `symetrique(p)` qui renvoie l'image du point instancié par symétrie par rapport au point p passé en paramètre.
- g. Ajouter une méthode `rotation_O(teta)` qui renvoie l'image du point instancié par rotation d'angle teta autour de l'origine (exprimé en radians)

[point.py](#)

Exo 6. Définir une classe Fraction pour représenter un nombre rationnel. Cette classe possède deux attributs, **num** et **denom**, entiers désignant le numérateur et le dénominateur. On demande que le dénominateur soit >0.

- a. Ecrire le constructeur de cette classe. Le constructeur doit lever une **ValueError** si le **denominateur** fourni n'est pas >0.
- b. Ajouter une méthode `__str__()` qui renvoie une chaîne de caractères de la forme « 12/35 », ou simplement 12 lorsque le dénominateur vaut 1.
- c. Ajouter les méthodes `__eq__(f2)` (pour **equals**) et `__lt__(f2)` (pour **less than**) qui prennent une deuxième fraction en argument et renvoient **True** si la fraction instanciée représente un nombre égal ou un nombre strictement inférieur à la deuxième fraction.
- d. Ajouter les méthodes `__add__` et `__mul__` qui prennent une deuxième fraction en argument et renvoient une nouvelle fraction représentant respectivement la somme et le produit des deux fractions.
- e. Ajouter une méthode `irreductible()` qui renvoie la fraction instanciée sous sa forme irréductible.

[fraction.py](#)

Exo 7. Dominos

Le jeu de dominos est constitué de 28 pièces toutes différentes : chaque pièce est constituée de deux côtés notés de 0 (blanc) à 6 points noirs. Il y a donc 7 dominos doubles (dont les deux côtés sont égaux).



1. Implémenter une classe **Domino** permettant de représenter les pièces avec ses deux côtés gauche et droite. Cette classe met à disposition :
 - a. Le constructeur qui initialise deux attributs gauche et droit.
 - b. Des méthodes pour tester si le domino est double (**est_un_double()**) ou blanc (**est_un_blanc()**), et une méthode qui renverra le nombre de points d'une pièce (**nb_points()**).
 - c. une méthode **__str__()** qui renvoie une chaîne décrivant le domino instancié.
Par ex. : **str(Domino(4,1))** renvoie : **"[4-1]"**



Tester toutes les méthodes de la classe en l'instanciant plusieurs fois (**intégrer ces tests au fichier**).

2. Implémenter une classe **Joueur** permettant de représenter le jeu de chaque joueur avec au moins ...
 - a. Le constructeur initialise deux attributs :
le nom du joueur,
le jeu du joueur sous la forme d'un tableau de dominos
 - b. La méthode **recevoir(dom)** qui rajoute le domino **dom** au jeu du joueur
 - c. La méthode **total()** qui renvoie le total des points du joueur.
 - d. une méthode **__str__()** qui renvoie une chaîne décrivant le jeu du joueur instancié.
Par ex., si **un_joueur** est une instance de la classe **Joueur** dont le nom est Tom :
str(un_joueur) renvoie : **"Tom : [6-4][4-2][4-3][4-4] --- total : 31"**



Tester la classe en l'instanciant plusieurs fois (**intégrer ces tests au fichier**).

3. Implémenter une classe **Jeu_Domino** permettant de manipuler le jeu de dominos complet
 - a. Le constructeur initialise au moins trois attributs :
le nombre de joueurs,
la pioche avec le jeu de dominos complet **et mélangé** sous forme d'un tableau de dominos (le nombre de pièces vaut 28 quand le jeu est complet),
les jeux de chaque joueur sous la forme d'un tableau de joueurs.



pour mélanger le jeu

(on peut utiliser la méthode **shuffle()** de la bibliothèque **random** de python
<https://docs.python.org/fr/3/library/random.html>)

- b. une méthode pour distribuer selon deux joueurs ou plus : dont on donne l'interface et le *docstring* :

```
def distribuer(self, nb_joueurs):
    """construit le jeu de chaque joueur en prélevant chaque pièce dans la
    pioche."""
```

4. une méthode pour afficher le jeu de chaque joueur ainsi que la pioche. Pour chaque joueur on affichera le nombre de points dans son jeu.



Ecrire le programme principal qui teste l'amorce du jeu en appelant chaque méthode de cette classe.

5. **Option** : Implémenter le jeu complet selon les règles suivantes :

- Mélanger le jeu complet. Distribuer le même nombre de dominos à chaque joueur, en laissant une pioche de quelques pièces ;
- Le premier joueur pose son plus fort domino, son voisin pose à l'une des extrémités un domino dont l'une des parties a le même nombre de points ;
- Chaque joueur joue à son tour et l'on constitue ainsi une chaîne ;
- Lorsqu'un joueur n'a pas de domino qui convient, il pioche une pièce du talon et passe son tour ;
- Le vainqueur est celui qui place le dernier de tous ses dominos.
- Si un joueur a tous les dominos de la même famille, par exemple tous les dominos comportant un 5, le vainqueur est celui qui a le plus petit nombre de points.

[domino.py](#) [jeu_domino.py](#)